DeployGuard FYP
DevOps Skill-Level Categorization

Junior, Intermediate, and Advanced concepts covered by the DeployGuard mini PaaS / IDP project

Prepared for student team discussion, FYP explanation, and skill ownership planning

Project context: DeployGuard is a mini Platform-as-a-Service / Internal Developer Platform that connects to GitHub repositories, scans Python/JavaScript apps, generates deployment contracts and Docker/ECS plans, runs an internal BullMQ pipeline, pushes images to ECR, plans/applies Terraform to AWS ECS/ALB, stores Terraform state in S3 with native lockfile, handles environment variables/secrets, and exposes pipeline events/recovery/debugging views.

How to read this document

- Junior level means concepts expected from a beginner DevOps/cloud engineer who can follow commands, understand logs, and operate basic tools.
- Intermediate level means concepts expected from someone who can design and integrate multiple systems: Docker, queues, AWS services, Terraform, backend APIs, and deployment workflows.
- Advanced level means platform-engineering depth: multi-stage orchestration, safety gates, contract-based deployment, state/secret management, runtime diagnostics, and production-grade failure handling.
- DeployGuard currently includes some advanced ideas in MVP/FYP form. Production-grade completeness still requires more hardening, multi-tenancy, rate limits, high availability, and deeper real-world testing.

Executive Summary

Area	Assessment	Explanation
Overall level	Intermediate to advanced FYP project	The project is beyond a simple CI/CD demo because it includes repository intelligence, deployment contracts, queues, AWS provisioning, state locking, environment variable handling, and runtime diagnostics.
Junior skills visible	Strong	Git, GitHub, Docker, basic Linux, environment variables, app logs, basic AWS resources, frontend/backend project structure.
Intermediate skills visible	Very strong	Docker templates, worker queues, ECS/ECR/ALB, Terraform plan/apply/destroy, S3 state, deployment pre-flight, project settings, backend APIs.
Advanced skills visible	Present but still maturing	PaaS-like deployability contract, secrets management, state backup before destroy, S3 native lockfile, ECS diagnostics, rollback logic, audit/recovery flows.
Best FYP positioning	Mini PaaS / IDP with DevOps automation	Present it as a platform that decides whether an app is deployable before creating AWS resources.

Skill-Level Matrix by Category

Category	Junior level	Intermediate level	Advanced / Platform level
Git/GitHub	clone, branch, commit, push, pull, PR basics, repo URL understanding	GitHub OAuth, repo metadata, branch selection, private repo access, commit SHA validation	GitHub App-style installation model, repo webhooks, token scoping, least-privilege source-control integration
Linux & terminal	basic shell, npm/pip commands, file paths, ports, logs	process management, environment files, permissions, service startup, debugging worker/API processes	automated bootstrap scripts, hardened runtime users, secure temp workspaces, sandboxing build contexts
Docker	Dockerfile basics, image build, container run, exposed ports	multi-stage Dockerfiles, static frontend nginx image, Node/Python server images, build context exclusions	template selection by deployment contract, secret-safe builds, local smoke tests, image provenance, reproducible builds
CI/CD & pipelines	understand stages: clone, build, scan, deploy	BullMQ worker, job lifecycle, retries, active-run prevention, pipeline events	idempotent jobs, resumable apply/destroy, failure recovery, canonical lifecycle, exactly-once-like operation safety
AWS basics	ECR, ECS, ALB, IAM, S3 names and purpose	ECS Fargate service, task definition, target group, CloudWatch logs, Secrets Manager	multi-account model, IAM least privilege, VPC endpoints/NAT strategy, stable release/rollback, cost and quota controls
Terraform/IaC	variables, plan, apply, destroy, providers	generated modules, S3 remote state, S3 native lockfile, apply approval, destroy flow	state backup/versioning, lock recovery, plan JSON analysis, drift detection, module versioning, safe destroy boundaries
Security	basic secret awareness, do not commit .env, vulnerability scan meaning	Trivy policies, secret masking, audit logs, auth guards, recent-auth before destroy	encrypted env vars, ECS secrets injection, policy-as-code, secret rotation, RBAC/workspace isolation, supply-chain controls
FinOps	understand AWS resources can cost money	mock cost gate, Infracost integration path, cost breakdown UI	budget policies, per-project quotas, forecast drift, live cost ingestion, chargeback/showback
Observability	logs, errors, status, health checks	pipeline events, ECS stopped task diagnostics, ALB target health, CloudWatch lines	structured logs, metrics, traces, SLOs, alert routing, Prometheus/Grafana integration, incident automation
Backend engineering	REST API basics, DTOs, env config, database migrations	NestJS modules/services/controllers, TypeORM entities, bulk APIs, validation, encryption service	transactional workflows, distributed locks, event sourcing, strong consistency for pipeline state, horizontal worker scaling
Frontend/product UX	forms, tables, status pages, buttons	project overview, pipeline page, settings, pre-flight recovery, env variable manager	progressive disclosure, simple/developer mode, recovery UX, mature PaaS onboarding, operational clarity
Platform engineering	deploying a single app	repo scanning, deployment contract, template mapping, cloud deployment orchestration	PaaS-level compatibility engine, service catalog, multi-tenant workspaces, self-service developer portal, policy-driven deployment

Module-by-Module Skill Categorization

Module	Skill level	What it proves	How to explain it
6.1 Auth & Audit	Junior + Intermediate	Email/GitHub auth, RBAC, audit events, logout/session behavior.	Advanced if extended to workspace membership, recent-auth policy, full audit compliance, and project-level authorization.
6.2 Projects & Workspace	Intermediate	Project creation, repo connection, branch/app-root settings, project env vars, deployment overrides.	Advanced if true multi-tenant workspace/invite model is completed.
6.3 Stack Detection	Advanced for FYP	Deep Python/JavaScript repo scanning, framework detection, app-root detection, env/port/start command inference.	This is one of the strongest platform-engineering features. It decides if a repo is deployable.
6.4 Templates & Pre-flight	Advanced for FYP	Dockerfile strategy, deployability gate, blockers/warnings, ECS runtime plan.	Mature platforms fail early before AWS; this module creates that behavior.
6.5 CI/CD & Queueing	Intermediate to Advanced	BullMQ worker pipeline, stages, retries, manual apply approval, events.	Advanced when jobs are idempotent, resumable, and protected against duplicate active operations.
6.6 Security Scanning	Intermediate	Trivy scan/policy engine exists but currently bypassed for demo.	Advanced if origin/fixability classification, policy-as-code, and secure supply-chain checks are enforced.
6.7 FinOps	Intermediate	Mock cost gate and Infracost integration path.	Advanced when real Infracost, budget rules, quotas, and cost policies are stable.
6.8 Infrastructure Provisioning	Intermediate to Advanced	Terraform for ECS/ALB/ECR/runtime infrastructure.	Advanced if modules are reusable, validated, drift-aware, and support safe rollout/rollback.
6.9 Distributed State	Advanced for FYP	S3 remote state, S3 native lockfile, state key per project, state backup before destroy.	Strong platform safety concept; production needs migration/recovery hardening.
6.10 Persistent Storage	Intermediate	EFS/storage module concept and detection of storage needs.	Advanced when storage is automatically provisioned only when app contract requires it.
6.11 ECS/Zero-Downtime/Rollback	Advanced for FYP	ECS Fargate, ALB, health validation, deployment circuit breaker, rollback concept.	Production-grade requires stable release registry, traffic shifting, rollback conditions, and deployment strategies.
6.12 Observability	Intermediate to Advanced	Pipeline events, ECS diagnostics, CloudWatch lines, ALB health.	Advanced when raw worker logs, metrics, alerts, traces, Prometheus/Grafana, and SLOs are completed.

What Each Team Member Should Understand

Role	Must know	Expected level
Everyone	Project goal, full deployment flow, why DeployGuard is not just GitHub Actions, why GitHub Actions is optional, basic Docker/AWS/Terraform terms.	Junior
Backend owner	NestJS modules, APIs, DeploymentContract, scanner, pre-flight, pipeline worker, env encryption, current-state API, auth guards.	Intermediate to Advanced
Frontend owner	Dashboard, project overview, pre-flight UI, env variable bulk form, pipeline status, recovery UX, simple vs developer mode.	Junior to Intermediate
Cloud/IaC owner	ECR, ECS Fargate, ALB, IAM roles, Secrets Manager, Terraform state in S3, lockfile, plan/apply/destroy, health checks.	Intermediate to Advanced
Security/QA owner	Secret handling, masking, Trivy bypass/current policy, audit logs, no secret leaks, test fixtures, pipeline failure scenarios.	Intermediate
Presenter/demo owner	Module mapping, user journey, why pre-flight blocks early, what is disabled for demo, remaining limitations.	Junior to Intermediate

Junior-Level Concepts in DeployGuard

- Git repository, branch, commit SHA, GitHub OAuth, and repository URL.
- Environment variables and why secrets should not be committed to GitHub.
- Basic Dockerfile, Docker image, Docker build, container port, and container logs.
- Basic pipeline stages: clone, detect, build, scan, push, plan, deploy.
- Basic AWS services used in the project: ECR stores images, ECS runs containers, ALB exposes traffic, S3 stores Terraform state.
- Basic Terraform commands: plan, apply, destroy, and why apply/destroy require confirmation.
- Basic UI states: setup required, running, paused, failed, live, destroyed.

Intermediate-Level Concepts in DeployGuard

- NestJS backend architecture: controllers, services, DTO validation, TypeORM entities, migrations, modules, guards.
- BullMQ worker pipeline: queued jobs, stages, retry, cancel, apply approval, destroy operation, pipeline events.
- Framework detection for JavaScript and Python apps: Vite/React, Next.js, Express, NestJS, FastAPI, Flask, Django, Streamlit.
- Docker template selection: static frontend with nginx, Node server, Python server, non-root runtime, production commands.
- ECS deployment plan: task definition, service, target group, listener, container port, health path, Secrets Manager references.
- Remote Terraform state: central S3 bucket, per-project state key, S3 lockfile, versioned backup before destroy.

- Environment variable workflow: bulk .env paste, encryption at rest, metadata-only API responses, runtime secret injection.
- Pre-flight blockers and warnings: missing env vars, ambiguous app root, unsupported language, no start command, port mismatch.

Advanced-Level Concepts in DeployGuard

- DeploymentContract / RuntimeContract as a single source of truth for Docker, ECS, ALB, pre-flight, current-state, and UI recovery.
- Fail-early PaaS behavior: block before Docker/ECR/Terraform when deployability is not proven.
- State safety: S3 native lockfile, state object versioning, backup reference before destroy, safe destroy boundaries.
- Secrets lifecycle: encrypt at rest, mask in UI, avoid build-layer leaks, inject runtime secrets through AWS Secrets Manager.
- Runtime diagnostics: ECS stopped task reason, exit code, service events, CloudWatch logs, ALB target health, sanitized troubleshooting.
- Policy-driven gates: security advisory/bypass mode now, future Trivy enforcement by fixability/origin; FinOps mock now, future real budget gates.
- Mature PaaS UX: Setup Required, Fix Deployment Setup, Ready to Deploy, Paused for Approval, Needs Attention, and Live only after health validation.

What Should Be Improved Next

Priority	Improvement	Why it matters	Urgency
1	Test DeploymentContract + pre-flight end-to-end	Use simple Vite/React and FastAPI repos before complex DB apps.	High
2	Complete secure env/secrets validation with AWS	Verify Secrets Manager permissions and ECS task secret injection.	High
3	Add local template smoke tests	Generated Dockerfile should build, run locally, and pass health check before AWS.	High
4	Persist raw worker stdout/stderr	Pipeline events are useful, but raw sanitized stage logs are needed for serious debugging.	Medium
5	Strengthen ECS diagnostics UI	Show exact app crash reason, CloudWatch lines, target health, and fix suggestion.	High
6	Enable real security policy later	Turn Trivy back on with advisory/fixability-based enforcement.	Medium
7	Enable real FinOps later	Infracost can become optional provider with mock fallback.	Medium
8	Workspace/team permissions	Add true workspace membership and project-level roles.	Medium
9	Prometheus/Grafana optional observability	Keep CloudWatch first; add Prometheus only after base deployment is stable.	Low/Medium

How to Present the Project Skill Level

- Do not present DeployGuard as only a CI/CD project. Present it as a mini PaaS / Internal Developer Platform.
- Say that the junior-level part is tool usage: Git, Docker, Linux, AWS basics, and Terraform basics.
- Say that the intermediate-level part is integration: NestJS backend, queues, Docker templates, AWS ECS/ECR/ALB, remote state, APIs, and frontend workflow.
- Say that the advanced-level part is platform engineering: DeploymentContract, deployability pre-flight, secrets management, safe state/destroy, and ECS diagnostics.
- Be honest that Security, FinOps, and Prometheus are currently disabled/mock/optional for demo stability, but the architecture supports them as modules.

One-Line Conclusion

DeployGuard is an intermediate-to-advanced FYP because it combines DevOps automation, cloud infrastructure, repository intelligence, secret handling, and PaaS-style deployment orchestration into one platform.